Iterator, sobre el material

El ejercicio 10.a del PPA *Publicidad Multipunto*. Cada hoja es el material **tal cual te lo da la cátedra**, y encima está marcado qué mirás, qué agarrás y qué tachás, con el porqué escrito al costado.

TE DAN

- La consigna
- La plantilla del CU
- El diagrama de clases
- El diagrama de comunicación

→

ENTREGÁS

- Clases con el patrón
- Diagrama de secuencia
- Pseudocódigo de cada método

CÓMO ESTÁN LAS MARCAS

resaltador amarillo la palabra que te avisa

violeta lo que se recorre adentro (programaciones)

rojo lo tachás

texto apagado no te sirve para este patrón

azul lo que se recorre afuera (puntos de emisión)

verde queda · lo agarrás

naranja lo nuevo que agregás / lo que entregás

así el porqué, escrito al costado

LOS PASOS

1. La consigna — hoja 2
2. La plantilla del CU — hoja 3
3. El diagrama de clases — hoja 5
4. El diagrama de comunicación — hoja 7
5. Agregás el patrón — hoja 9
6. Secuencia y pseudocódigo — hoja 10
7. El Iterator en una hoja — hoja 12

LAS 5 PREGUNTAS QUE LE HACÉS A CUALQUIER ENUNCIADO

1. ¿Qué palabra me dice que hay que **recorrer**? (recorrer, por cada, busca todos…)
2. ¿Qué **colecciones** se recorren? ¿Una dentro de la otra (**«a su vez»**)?
3. ¿**Quién tiene** cada colección? Ese es el **contexto** → va a IAgregado.
4. ¿Qué **condición filtra** cada recorrido? → va en `cumpleFiltro()`.
5. ¿Qué **pregunta final** tiene que responder la dinámica?

LA IDEA

No hace falta leer todo. Vas a buscar tres cosas: **qué se recorre** (las colecciones), **quién tiene cada colección** (los contextos) y **qué condición filtra** cada recorrido. Todo lo demás se tacha.

1 La consigna

Así te la dan. Encima: lo que buscás con la vista, lo que tachás y por qué.

Universidad Tecnológica Nacional – Facultad Regional Córdoba
Cátedra de Diseño de Sistemas de Información

5. Mencione los patrones de asignación de responsabilidades utilizados, identificando los números de los mensajes que intervienen en la aplicación del patrón. Justifique la elección cada patrón.
6. Realizar el DTE de la clase Punto de Emisión, identificando estados, transiciones, métodos, casos de uso y condiciones de control. *5 a 9: no son de patrones → no los leés*
7. Identificar los requerimientos no funcionales y analizar su significancia para la arquitectura.
8. Identificar los patrones arquitectónicos relevantes y construir una representación gráfica de los mismos.
9. Construir las siguientes vistas arquitectónicas, definiendo para cada vista el punto de vista asociado:
    a. Vista Arquitectónica de la funcionalidad.
    b. Vista Arquitectónica de Diseño: Componentes e Interfaces.
    c. Vista Arquitectónica del Despliegue: Nodos y subsistemas principales.

↓ el de patrones

10. Realice la vista de estructura (diagrama de clases) y la vista dinámica (diagrama de secuencia) de los dos patrones de diseño que resuelvan las siguientes consideraciones. Especificar el comportamiento de cada uno de los métodos utilizados en la dinámica, mediante una descripción textual o pseudocódigo: *← lo subrayado en naranja: las 3 cosas que entregás*
    a. Implementar un mecanismo que permita recorrer los puntos de Emisión y a su vez sus programaciones. Modelar la dinámica para encontrar los puntos de emisión que tienen la tanda actualizada. *← lo que busca la dinámica (buscalo en el diagrama de clases)*
    b. Resolver la forma en que el sistema notificará la modificación de una tanda publicitaria a los distintos puntos de emisión y a la pantalla de vista previa de una programación que posea esa tanda publicitaria. Modelar la dinámica que muestra la modificación sobre ambas pantallas (Punto de emisión y Vista Previa). *← b: «notificará» = Observer. No es este*
11. Mapear las clases a un modelo de datos relacional. *11 a 13: tampoco*
12. Enumerar los casos de uso que se deberán agregar y/o modificar para contemplar la nueva funcionalidad detallada a continuación:
    Dado que la empresa desea cambiar la definición de la tarifa de los servicios, se deberá adecuar el sistema para que se pueda realizar el cálculo del costo de los servicios contratados. Las necesidades del negocio a cubrir son:
    - Definir el nivel de afluencia de público según la ubicación del punto de emisión (en base al promedio de cantidad de personas por hora)
    - Agregar Categorías al punto de emisión según la descripción de características de la pantalla física tales como dimensiones de la pantalla, definición y visibilidad.
    - La tarifa del servicio ahora es dependiente de ambas cosas mencionadas anteriormente.
    - Además, debe manejar 3 fechas de vencimientos con porcentajes de aumento diferenciado para cada uno, según una cantidad determinada de días desde la fecha de fin del periodo de reproducción.
13. Realizar los cambios correspondientes en el diagrama de dominio provisto.

permita recorrer → ITERATOR. También te avisan «por cada», «todos los...»

y a su vez → anidado: por cada punto de emisión, sus programaciones

los puntos de Emisión → lo que recorrés afuera → IteradorPuntoEmision

sus programaciones → lo de adentro → IteradorProgramacion. «sus»: son del PE

2 La plantilla del CU 14

Así te la dan. Las filas que importan están marcadas; el resto, apagado o tachado.

Universidad Tecnológica Nacional – Facultad Regional Córdoba
Cátedra de Diseño de Sistemas de Información

Especificación de los Casos de Uso afectados al desarrollo del Ejercicio

Nombre del Caso de uso: Generar programación de tandas publicitarias	**Nro. de Orden**: 14
Prioridad: ☒ Alta ☐ Media ☐ Baja	
Complejidad: ☐ Simple ☐ Mediano ☒ Complejo ☐ Muy Complejo ☐ Extremadamente Complejo	
Actor Principal: Administrador de Programación (AP)	**Actor Secundario**: No Aplica
Tipo de Caso de uso: ☒ Concreto ☐ Abstracto	
Objetivo: Generar la programación de tandas publicitarias para un punto de emisión en base a los servicios contratados para un determinado periodo. ← *CU 14: el de los diagramas que siguen*	

Flujo Básico ← *leé las filas «Sistema:» y buscá verbos que recorren: busca, todos, valida*

1. **AP:** El caso de uso comienza cuando selecciona la opción "Generar Programación".
2. **Sistema:** Busca y muestra todos los puntos de emisión que se encuentran habilitados y solicita se seleccione el punto de emisión para el cual se generará la programación. ↑ *habilitados = el filtro*
3. **AP:** selecciona el punto de emisión para el cual desea generar una programación.
4. **Sistema:** Solicita se ingrese el período para el cual se quiere generar la programación
5. **AP:** Ingresa la fecha de inicio y la fecha de fin para la programación.
6. **Sistema:** valida que el punto de emisión no posea una programación asociada para el periodo ingresado y no existe. (Ver **RN Validación Periodo Programación**) ↑ *= recorrido de adentro*
7. **Sistema:** ~~Busca todos los servicios contratados~~ para el punto de emisión seleccionado, cuyo período de reproducción coincida con el período ingresado y encuentra al menos uno. **(ver observación 2)** Por cada servicio contratado muestra el nombre, el código y la duración de la publicidad asociada al servicio, las fechas en que debería reproducirse, el tipo de servicio y el cliente asociado, y solicita se seleccionen las publicidades que se desean incluir en la programación. ✕ *no está en la consigna*
8. **AP:** Selecciona las publicidades a incluir en la programación. ↓ ✕ *tandas: tampoco*
9. **Sistema:** ~~Busca y muestra~~ el nombre y descripción de ~~todas las tandas~~ que contengan las publicidades seleccionadas y no hayan sido dadas de baja y encuentra al menos una, por lo cual solicita se seleccionen las tandas a incluir en la programación y el orden en el cual se van a reproducir.
10. **AP:** Selecciona las tandas que desea incluir y el orden de reproducción de cada tanda.
11. **Sistema:** Solicita se ingrese el nombre de la programación.
12. **AP:** ingresa el nombre de la programación a crear. *filas apagadas: acá no se recorre nada*
13. **Sistema:** Solicita confirmar para generar la programación.
14. **AP**: Confirma la generación de la programación.
15. **Sistema**: Genera la nueva programación para el punto de emisión seleccionado, con fecha de inicio y de fin igual al período ingresado, con fecha de creación igual a la fecha actual, con las tandas seleccionadas en el orden de reproducción indicado.

~~**Flujos Alternativos**~~ ✕ *alternativos: no cambian el patrón (siguen en la hoja que viene)*

Busca y muestra todos los puntos de emisión → recorrido de afuera: IteradorPuntoEmision

habilitados: no sirven todos → es el filtro: estaHabilitado()

valida que no tenga una programación... → mira todas las programaciones de ese PE (adentro)

~~servicios~~ y ~~tandas~~ también se recorren, pero la consigna no los nombra

2 La plantilla del CU (sigue)

La hoja que viene después en el material. Para el Iterator no hay nada para agarrar acá.

Universidad Tecnológica Nacional – Facultad Regional Córdoba
Cátedra de Diseño de Sistemas de Información

A1. Sistema: No encuentra puntos de emisión habilitados.
A2. Sistema: No encuentra ningún servicio contratado cuyo período de reproducción coincida.
A3. Sistema: El punto de emisión ya tiene una programación para el período ingresado.
A4. Sistema: El punto de emisión no posee servicios contratados para el período ingresado.
A5: Sistema: No existen tandas que contengan las publicidades seleccionadas.
A6. AP: No confirma la operación.
A7: AP: Cancela el caso de uso. ← alternativos y observaciones: no cambian el patrón

Observaciones:
1. El AP puede cancelar la operación en cualquier momento.
2. El periodo de reproducción del servicio concuerda con el periodo seleccionado o coincide al inicio o al final del mismo.

CU 11 · Actualizar tanda: es la base de la 10.b (Observer). Para el Iterator, ✕

Nombre del Caso de uso: Actualizar tanda publicitaria	**Nro. de Orden**: 11
Prioridad: ☒ Alta ☐ Media ☐ Baja	
Complejidad: ☐ Simple ☐ Mediano ☒ Complejo ☐ Muy Complejo ☐ Extremadamente Complejo	
Actor Principal: Administrador de Programación (AP)	**Actor Secundario**: No Aplica
Tipo de Caso de uso: ☒ Concreto ☐ Abstracto	
Objetivo: Actualizar una tanda publicitaria incorporando o quitando detalles de tanda publicitaria.	

Flujo Básico

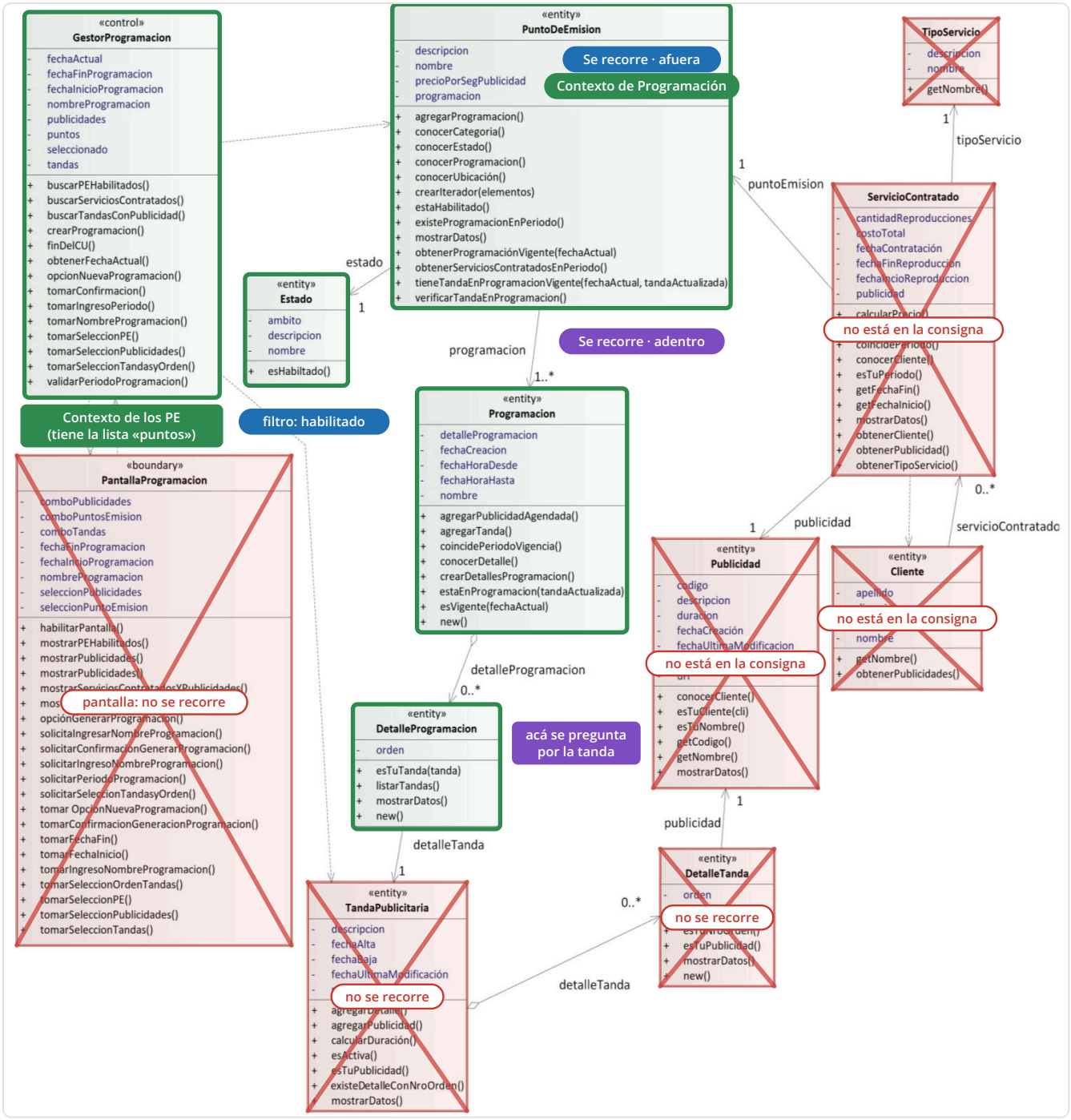
1. **AP:** El caso de uso comienza cuando selecciona la opción "Actualizar Tanda publicitaria".
2. **Sistema:** busca y lista por nombre todas las tandas publicitarias existentes que no hayan sido dadas de baja y permite se seleccione una para actualizarla.
3. **AP:** Selecciona la tanda publicitaria que desea actualizar.
4. **Sistema:** Muestra para la tanda publicitaria seleccionada el nombre, la descripción y lista todos sus detalles con cada publicidad y su número de orden, permitiendo agregar o eliminar detalles de publicidades.
5. **AP:** Selecciona la opción *Agregar Publicidad*.
6. **Sistema:** Solicita se ingrese el periodo dentro del cual se contrató la publicidad a incluir en la tanda publicitaria.
7. **AP:** Ingresa la fecha desde y la fecha hasta para buscar las publicidades.
8. **Sistema:** Busca todos los servicios contratados cuyo período de reproducción coincida con el período ingresado y encuentra al menos uno. **(ver Observación 2)**
   Por cada servicio contratado busca todas las publicidades que fueron contratadas, mostrando el nombre, el código y la duración para cada publicidad.
   Además, muestra el cliente que contrató el servicio y solicita que se seleccione la/s publicidad/es que se desea incluir en la tanda publicitaria y el orden de cada publicidad en la tanda.
9. **AP:** Selecciona la/s publicidad/es a incluir en la tanda publicitaria y el orden.

¿POR QUÉ EL CU 11 VIENE EN EL MATERIAL?

Porque la consigna 10 tiene dos letras. La 10.a (Iterator) encuentra los puntos de emisión que tienen la tanda actualizada; la 10.b (Observer) les avisa del cambio. El CU 11 es el que actualiza la tanda. Para la 10.a solo usás el CU 14.

3 El diagrama de clases

Tachás todo lo que no sea las dos colecciones y quien las tiene. Lo que queda es el esqueleto del patrón.



1. **Las clases que nombra la consigna quedan:** **PuntoDeEmision** (se recorre afuera) y **Programacion** (se recorre adentro).
2. **Quién TIENE cada lista es el contexto:** el Gestor tiene `puntos` → contexto de los PE. El PE tiene `programacion` (1..*) → contexto de Programación.
3. **Las que necesita el filtro o la pregunta final también quedan:** Estado (habilitado) y DetalleProgramacion (la tanda).
4. **Lo demás, X:** la pantalla (boundary) nunca va; servicios, publicidades, clientes y tipos no están en la consigna; la tanda te llega como dato, no se recorre.

MIRÁS

- las clases que nombra la consigna
- quién tiene cada lista (atributo o 1..*)

AGARRÁS

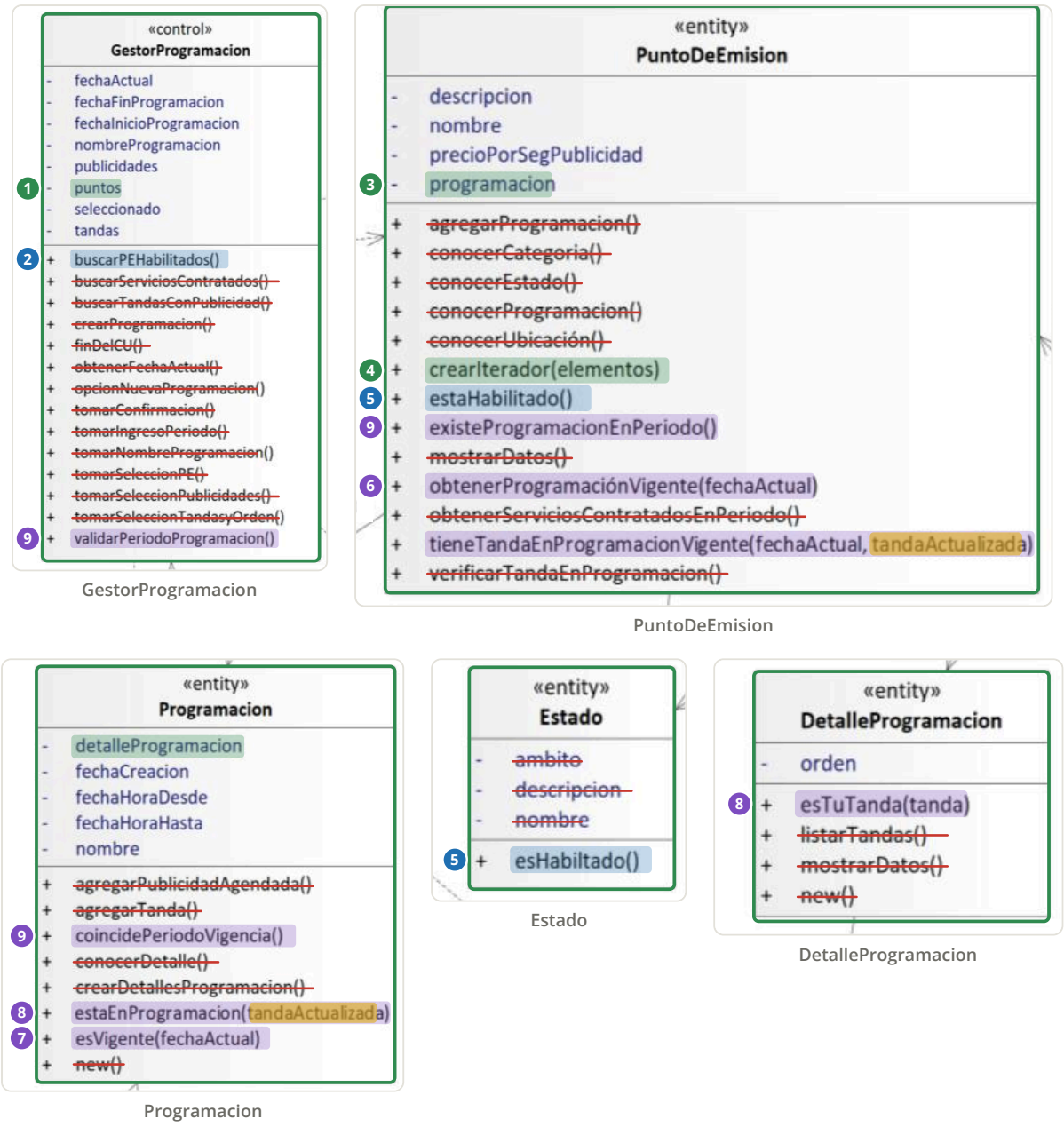
- Gestor: contexto de los PE
- PuntoDeEmision: se recorre y es contexto
- Programacion: se recorre
- Estado y DetalleProgramacion

TACHÁS

- PantallaProgramacion
- servicios, publicidades, clientes, tipos
- TandaPublicitaria y DetalleTanda

3 El diagrama de clases, de cerca

Ahora los métodos: marcás los que recorren, filtran y unen una clase con la otra. El resto, raya.



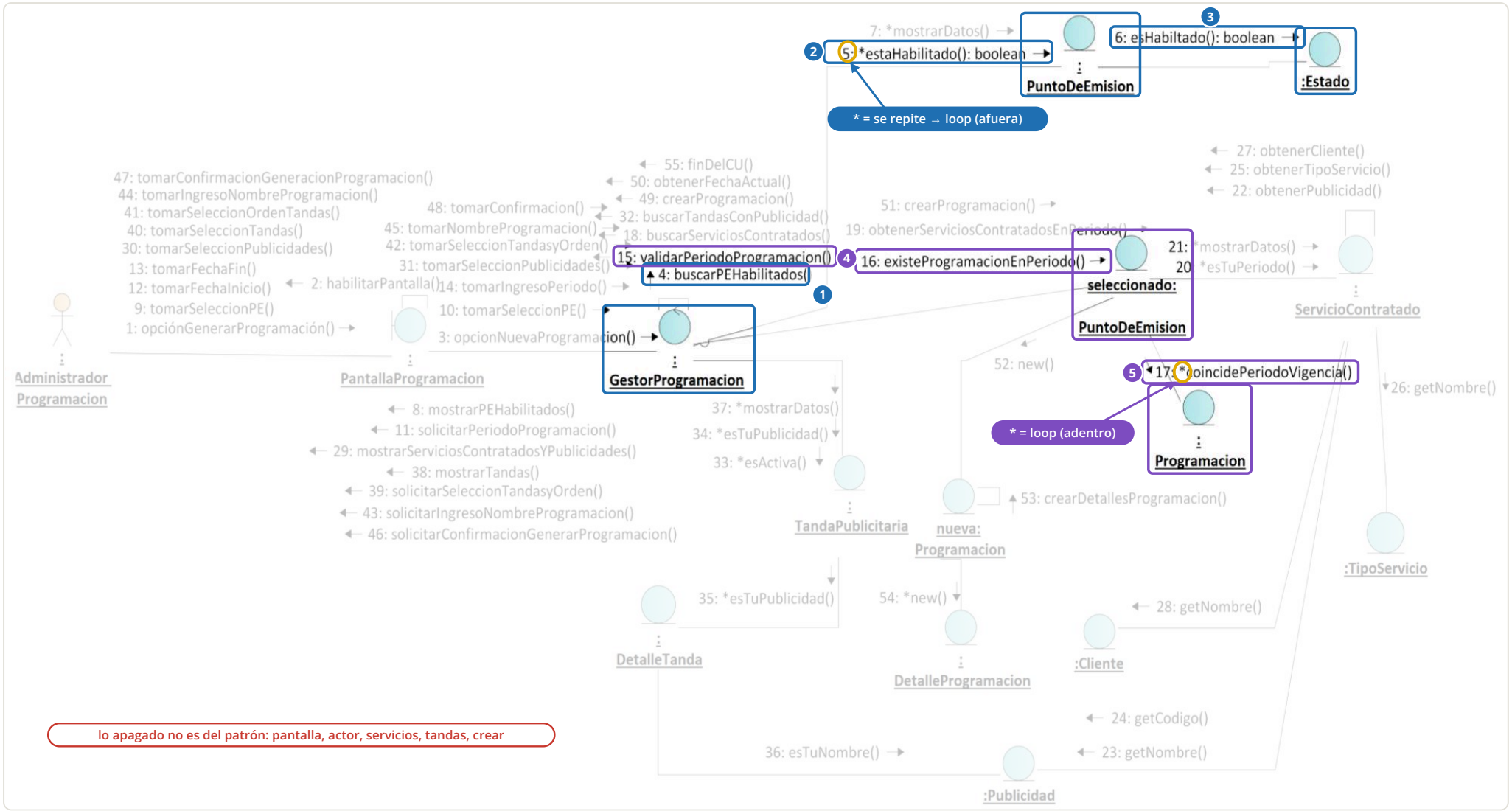
1. `puntos`: la lista de PE. Quien tiene la lista es el contexto → el Gestor va a realizar IAgregado.
2. `buscarPEHabilitados()`: de acá sale el recorrido de afuera (mensaje 4).
3. `programacion` (1..*): la lista de programaciones → el PE también es contexto.
4. `crearIterador(elementos)` ya viene dibujado en el PE: es la pista de que es IAgregado.
5. `estaHabilitado()` → `esHabilitado()` de Estado: el filtro de afuera («habilitados» del CU).
6. `tieneTandaEnProgramacionVigente(fechaActual, tandaActualizada)`: ¡la frase de la consigna como parámetro! Es el método que une PE con Programación. Adentro usa `obtenerProgramaciónVigente(fechaActual)`.
7. `esVigente(fechaActual)`: el filtro de adentro.
8. `estaEnProgramacion(tandaActualizada)` → `esTuTanda(tanda)` de cada detalle: la pregunta final.
9. `validarPeriodoProgramacion()` → `existeProgramacionEnPeriodo()` → `coincidePeriodoVigencia()`: la misma forma (el PE recorre sus programaciones) pero para el CU 14. Te sirven para reconocer el recorrido en el diagrama de comunicación.

TRUCO

Buscá en los métodos las palabras de la consigna. «tanda actualizada» aparece tal cual como parámetro → esos son los métodos de tu dinámica.

4 El diagrama de comunicación

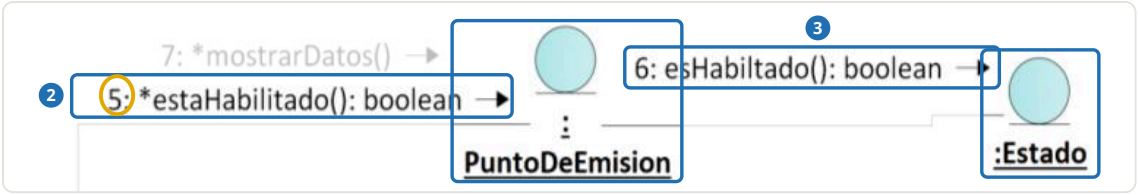
Buscás los ***** (se repite = loop) y quién le habla a quién. Lo que no toca a las dos colecciones queda apagado.



recorrido de afuera: mensajes 4·5·6 · recorrido de adentro: mensajes 15·16·17 · el * que marca el loop · los números ①–⑤ se explican en la hoja siguiente

4 El diagrama de comunicación, de cerca

Leés cada mensaje marcado y lo pasás al patrón.



arriba a la derecha: el PE y su Estado



el centro: el Gestor, el PE seleccionado y sus programaciones

	Mensaje	Qué te dice	Qué hacés
1	`4: buscarPEHabilitados()` Gestor → sí mismo	Arranca el recorrido de afuera.	El Gestor crea el **IteradorPuntoEmision**.
2	`5: *estaHabilitado()` Gestor → PE	El ***** = por cada PE → **loop de afuera**. estaHabilitado es el filtro.	Va en `cumpleFiltro()` del IteradorPuntoEmision.
3	`6: esHabilitado()` PE → Estado	Lo que el filtro pregunta por dentro.	Queda igual, dentro del loop.

4	`15: validarPeriodoProgramacion()`
	`16: existeProgramacionEnPeriodo()`
	Gestor → PE seleccionado
		Le pide al PE que mire **sus** programaciones.	El PE (contexto) crea el **IteradorProgramacion**.

5	`17: *coincidePeriodoVigencia()` PE → Programacion	El ***** = **loop de adentro**.	La condición va en `cumpleFiltro()` del IteradorProgramacion.

OJO

Este diagrama es del CU 14: te muestra **la forma** (quién recorre a quién y con qué filtro). Los métodos de «tanda actualizada» los sacás del diagrama de clases (hoja 6, 6·7·8). La forma es la misma.

MIRÁS

- los ***** (loop)
- quién le habla a quién

AGARRÁS

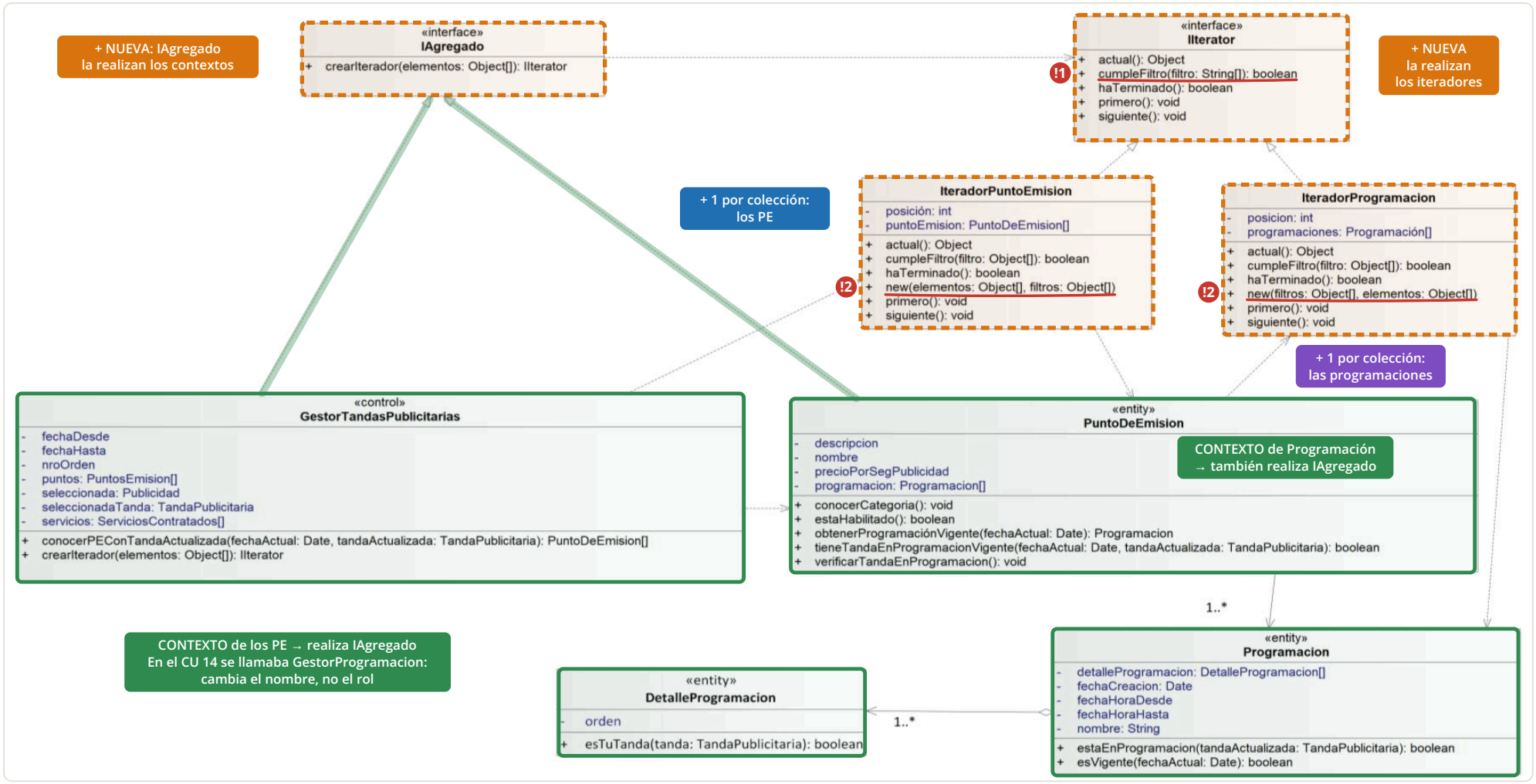
- 4·5·6 → afuera
- 15·16·17 → adentro

TACHÁS

- actor y Pantalla
- servicios (18–28)
- tandas (32–38)
- crear programación (49–55)

5 Agregás el patrón

A lo que quedó del paso 3 le sumás 2 interfaces y 1 iterador por cada colección. Así queda la solución de la cátedra.



2 interfaces, siempre

IAgregado: `crearIterador(elementos: Object[]): IIterator`

IIterator: `primero()`, `siguiente()`, `haTerminado()`, `actual()`, `cumpleFiltro(filtros)`

1 iterador por colección

Dos colecciones → **IteradorPuntoEmision** e **IteradorProgramacion**. Cada uno: `posicion` + la lista, `new(elementos, filtros)` y los 5 de IIterator.

Los contextos van al IAgregado

El Gestor (tiene los PE) y PuntoDeEmision (tiene las programaciones) realizan IAgregado y tienen `crearIterador()`.

Lo que ya tenías

PuntoDeEmision, Programacion y DetalleProgramacion, solo con los métodos que marcaste en la hoja 6.

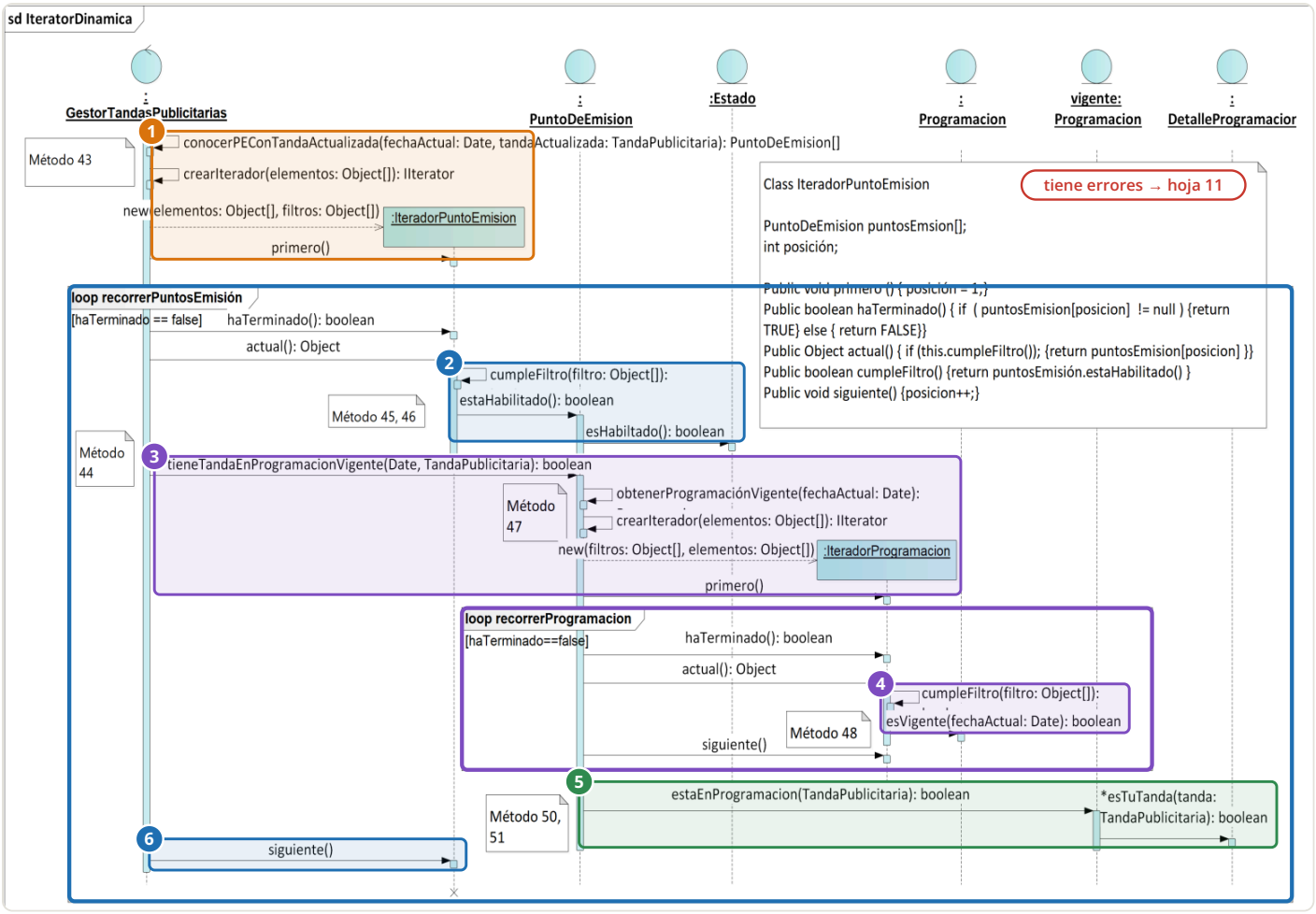
No copies esto

!1 la interfaz dice `String[]` y los iteradores `Object[]` → Object[] en todos.

!2 `new(…)` con los parámetros al revés en cada iterador → el mismo orden en los dos.

6 La secuencia (solución)

Cada loop es un iterador. Los pasos son siempre los mismos; lo que cambia es el filtro y la pregunta del medio.



1. **Arranque** (método 43): el Gestor crea el iterador de PE → `crearIterador()`, `new(elementos, filtros)`, `primero()`.
2. **Loop de afuera**: `actual()` → `cumpleFiltro()` → `estaHabilitado()` → `esHabilitado()`. Es el «habilitados» del CU (mensajes 5·6).
3. Por cada PE: `tieneTandaEnProgramacionVigente()`. El PE es contexto: arma **su** iterador de programaciones (`crearIterador`, `new`, `primero`).
4. **Loop de adentro**: busca la programación vigente → `cumpleFiltro()` → `esVigente(fechaActual)`.
5. **La pregunta de la consigna**: a la vigente, `estaEnProgramacion(tandaActualizada)` → `*esTuTanda()` en cada detalle.
6. `siguiente()`: pasa al próximo PE.

EL MOLDE (SIEMPRE IGUAL)

```
// en el contexto
crearIterador(elementos)
new(elementos, filtros)   → :IteradorX
primero()
loop [haTerminado() == false]
    actual()
    cumpleFiltro()  → el método filtro
    … lo que pide la consigna …
    siguiente()
```

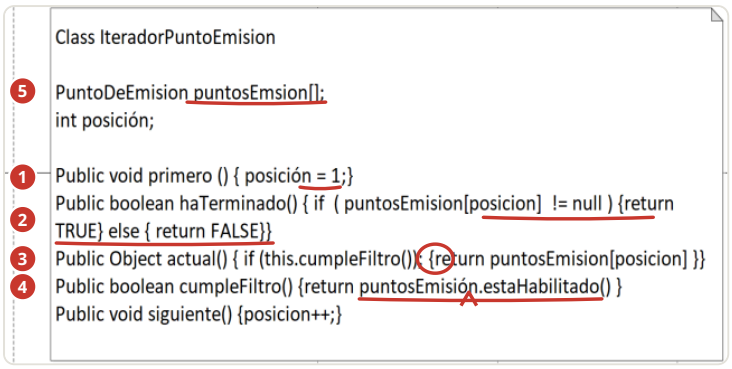
Las notitas «Método 43… 51» marcan los métodos que después describís en pseudocódigo.

DE TUS MARCAS A LA SECUENCIA

En el diagrama de comunicación (CU 14)		En la secuencia de la solución
`4: buscarPEHabilitados()`	1	arranque: `conocerPEConTandaActualizada()` (método 43) crea el iterador de PE
`5: *estaHabilitado()` · `6: esHabilitado()`	2	el mismo filtro, ahora dentro de `cumpleFiltro()`
`16: existeProgramacionEnPeriodo()`	3	`tieneTandaEnProgramacionVigente()`: el PE arma su iterador
`17: *coincidePeriodoVigencia()`	4	`esVigente(fechaActual)` dentro de `cumpleFiltro()`
no está en el CU 14: sale de «tanda actualizada»	5	`estaEnProgramacion(tandaActualizada)` → `*esTuTanda()`

6 El pseudocódigo (solución)

A la izquierda, lo de la cátedra con los errores marcados. A la derecha, cómo queda bien.



```
Class IteradorPuntoEmision

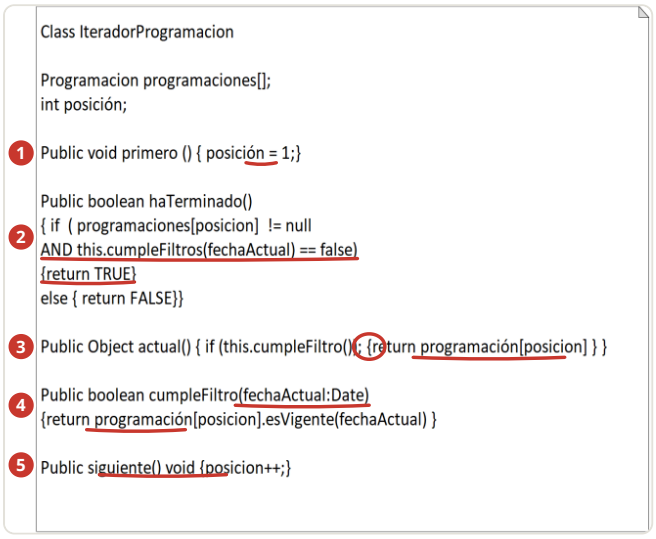
PuntoDeEmision puntosEmsion[];
int posición;

Public void primero () { posición = 1;}
Public boolean haTerminado() { if  ( puntosEmision[posicion]  != null ) {return
TRUE} else { return FALSE}}
Public Object actual() { if (this.cumpleFiltro()){ {return puntosEmision[posicion] }}
Public boolean cumpleFiltro() {return puntosEmisión.estaHabilitado() }
Public void siguiente() {posicion++;}
```

cátedra · IteradorPuntoEmision

```
class IteradorPuntoEmision implements IIterator
  PuntoDeEmision[] puntosEmision
  Object[] filtros
  int posicion

  new(Object[] elementos, Object[] filtros) {
    puntosEmision = elementos; this.filtros = filtros }
  void primero() { posicion = 0 }
  boolean haTerminado() {
    return posicion >= puntosEmision.length }
  Object actual() {
    if (cumpleFiltro(filtros))
      return puntosEmision[posicion]
    return null }
  boolean cumpleFiltro(Object[] filtros) {
    return puntosEmision[posicion].estaHabilitado() }
  void siguiente() { posicion++ }
```



```
Class IteradorProgramacion

Programacion programaciones[];
int posición;

Public void primero () { posición = 1;}

Public boolean haTerminado()
{ if  ( programaciones[posicion]  != null
AND this.cumpleFiltros(fechaActual) == false)
{return TRUE}
else { return FALSE}}

Public Object actual() { if (this.cumpleFiltro()); {return programación[posicion] } }

Public boolean cumpleFiltro(fechaActual:Date)
{return programación[posicion].esVigente(fechaActual) }

Public siguiente() void {posicion++;}
```

cátedra · IteradorProgramacion

```
class IteradorProgramacion implements IIterator
  Programacion[] programaciones
  Object[] filtros      // filtros[0] = fechaActual
  int posicion

  new(Object[] elementos, Object[] filtros) {
    programaciones = elementos; this.filtros = filtros }
  void primero() { posicion = 0 }
  boolean haTerminado() {
    return posicion >= programaciones.length }
  Object actual() {
    if (cumpleFiltro(filtros))
      return programaciones[posicion]
    return null }
  boolean cumpleFiltro(Object[] filtros) {
    return programaciones[posicion]
              .esVigente((Date) filtros[0]) }
  void siguiente() { posicion++ }
```

1. `primero()` pone `posicion = 1`: en un arreglo el primero es el **0**; con 1 te salteás el primer elemento.
2. `haTerminado()` está **al revés**: da TRUE cuando todavía hay elemento, y el loop ni arranca. Tiene que dar TRUE cuando ya no queda ninguno. En IteradorProgramacion además mete el filtro: el loop cortaría en la primera que no cumple.
3. `actual()`: `if (…);` — el punto y coma deja el if vacío. Y no dice qué devuelve si no cumple.
4. `cumpleFiltro()`: le falta `[posicion]` (le pregunta al arreglo, no al elemento). En IteradorProgramacion la firma no es la de la interfaz.
5. Nombres: `puntosEmsion` / `puntosEmisión` / `puntoEmision`, `programación` / `programaciones`, `cumpleFiltros` / `cumpleFiltro`. Elegí uno y usalo igual en clases, secuencia y pseudocódigo.

LOS DEMÁS MÉTODOS DE LA DINÁMICA, EN DESCRIPCIÓN TEXTUAL

`conocerPEConTandaActualizada(fechaActual, tandaActualizada)` Gestor · método 43	Crea el IteradorPuntoEmision con todos los PE. Mientras no haya terminado: toma el actual (si está habilitado), le pregunta `tieneTandaEnProgramacionVigente()` y, si da true, lo guarda. Devuelve los PE guardados.
`tieneTandaEnProgramacionVigente(fechaActual, tandaActualizada)` PuntoDeEmision	Busca su programación vigente con `obtenerProgramaciónVigente()` y le pregunta `estaEnProgramacion(tandaActualizada)`.
`obtenerProgramaciónVigente(fechaActual)` PuntoDeEmision	Crea el IteradorProgramacion con sus programaciones y la fecha como filtro; recorre hasta encontrar la que cumple `esVigente()` y la devuelve.
`esVigente(fechaActual)` Programacion	True si la fecha actual está entre `fechaHoraDesde` y `fechaHoraHasta`.
`estaEnProgramacion(tanda)` · `esTuTanda(tanda)` Programacion · DetalleProgramacion	Recorre sus detalles y devuelve true si alguno responde `esTuTanda(tanda)`, es decir, si su tanda es la actualizada.

El Iterator en una hoja

Para el parcial: el mismo orden, con cualquier enunciado.

TE AVISA QUE ES ITERATOR

recorrer · por cada · busca (y muestra) todos · a su vez → anidado · los que cumplen / habilitados → filtro · * en el diagrama · atributo lista o 1..*

Hoja	Mirás	Agarrás	Tachás
1 · Consigna	el ítem 10 y la letra del Iterator	**recorrer** → Iterator · las colecciones (afuera / adentro) · «a su vez» = anidado · lo que buscás	la letra del Observer
2 · Plantilla CU	las filas «Sistema:»	«Busca / todos / valida» sobre las colecciones de la consigna · el filtro («habilitados»)	filas del AP · otros recorridos (servicios, tandas) · alternativos
3 · Clases	quién tiene cada lista (atributo o 1..*) · métodos con palabras de la consigna	contextos (Gestor, PE) · lo recorrido (PE, Programación) · métodos filtro · métodos que unen	Pantalla · clases fuera de la consigna · tomar / mostrar / conocer / agregar
4 · Comunicación	los ***** y quién le habla a quién	mensajes del recorrido de afuera y del de adentro	actor, pantalla y los mensajes de otras clases
5 · Armás clases	—	**2 interfaces + 1 iterador por colección** · contextos → IAgregado · iteradores → IIterator	—
6 · Secuencia y pseudo	—	el molde por cada loop · filtro en `cumpleFiltro()` · la pregunta de la consigna en el medio	los errores de la hoja 11

LO QUE SIEMPRE DIBUJÁS

- **IAgregado** con `crearIterador(elementos: Object[]): IIterator`.
- **IIterator** con `primero()`, `siguiente()`, `haTerminado()`, `actual()`, `cumpleFiltro(filtros: Object[])`.
- Un **IteradorX** por colección: `posicion`, la lista, `new(elementos, filtros)`.
- Cada contexto realiza IAgregado (- - -▷); cada iterador realiza IIterator y depende de lo que recorre (- - ->).
- Anidado: el loop de adentro lo arma el contexto de adentro (acá, el PE), dentro del loop de afuera.

EL MOLDE DE CADA LOOP

```
// en el contexto
crearIterador(elementos)
new(elementos, filtros)   → :IteradorX
primero()
loop [haTerminado() == false]
    actual()
    cumpleFiltro()  → el método filtro
    … lo que pide la consigna …
    siguiente()
```

LO QUE NO TENÉS QUE HACER (ESTÁN EN LA SOLUCIÓN DE LA CÁTEDRA O EN TUS APUNTES)

- `haTerminado()` al revés: tiene que dar **true cuando ya no quedan** elementos.
- `primero()` con `posicion = 1`: el primero de un arreglo es el **0**.
- Mezclar el filtro en `haTerminado()`: el filtro va **solo** en `cumpleFiltro()`.
- `cumpleFiltro()` sobre la lista entera: preguntale al elemento, `lista[posicion]`.
- `crearIterador()` que devuelve void: devuelve el **IIterator** que crea.
- **En tus apuntes:** `Segundo()` → `siguiente()` · `crearIterador(…): Void` → devuelve `IIterator` · en IteradorProgramacion el atributo es `programaciones: Programacion[]`, no `PuntosDeEmision[]`.
- Firmas distintas entre interfaz e iteradores (`String[]` / `Object[]`, orden de `new`) o nombres que cambian de un diagrama a otro.